

2 Introduction aux React Hooks

2.1 Qu'est-ce qu'un Hook ?

Un **Hook** est une fonction spéciale React qui permet d'utiliser des fonctionnalités React (state, lifecycle, context) dans des composants fonctionnels.

△ Important

Convention de nommage :

Tous les Hooks commencent par **use** : `useState`, `useEffect`, `useContext`, `useRef`...

2.2 Pourquoi les Hooks ?

Avant React 16.8 (2019) :

Listing 2 – Class component (ancienne méthode)

```
// Avant : Class components pour state
class Counter extends React.Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
  }

  increment = () => {
    this.setState({ count: this.state.count + 1 });
  }

  render() {
    return (
      <div>
        <p>{this.state.count}</p>
        <button onClick={this.increment}>+</button>
      </div>
    );
  }
}
```

Après React 16.8 (avec Hooks) :

Listing 3 – Functional component avec Hooks

```
// Maintenant : Functional components + Hooks
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>{count}</p>
      <button onClick={() => setCount(count + 1)}>+</button>
    </div>
  );
}
```

Avantages des Hooks :

— Code plus court et lisible

- Pas de `this` (source d'erreurs)
- Logique réutilisable (custom hooks)
- Meilleure performance

2.3 Règles des Hooks

△ Important

2 règles absolues :

Règle 1 : Appeler Hooks uniquement au top-level

```
// INTERDIT - Dans condition
function MyComponent() {
  if (condition) {
    const [state, setState] = useState(0); // ERREUR
  }
}

// INTERDIT - Dans boucle
function MyComponent() {
  for (let i = 0; i < 10; i++) {
    const [state, setState] = useState(0); // ERREUR
  }
}

// CORRECT - Top-level
function MyComponent() {
  const [state, setState] = useState(0); // OK

  if (condition) {
    // Utiliser state ici est OK
  }
}
```

Règle 2 : Appeler Hooks uniquement dans composants React ou custom hooks

```
// INTERDIT - Fonction normale
function calculateTotal() {
  const [total, setTotal] = useState(0); // ERREUR
}

// CORRECT - Composant React
function MyComponent() {
  const [total, setTotal] = useState(0); // OK
}

// CORRECT - Custom Hook
function useTotal() {
  const [total, setTotal] = useState(0); // OK
  return total;
}
```

Pourquoi ces règles ?

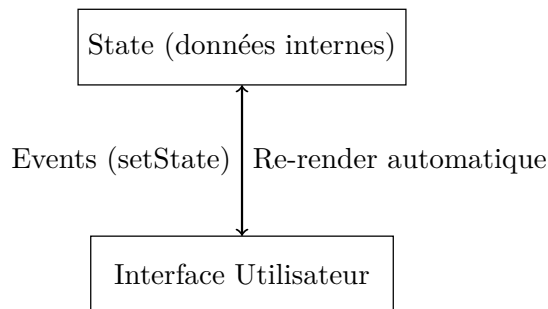
React s'appuie sur l'ordre d'appel des Hooks pour maintenir l'état correct entre les renders. Si l'ordre change (condition, boucle), React perd le fil.

3 State avec useState

3.1 Qu'est-ce que le State ?

Le **state** est une variable spéciale React qui :

- Peut changer au fil du temps
- Déclenche un re-render quand elle change
- Est gérée localement dans le composant



3.2 Syntaxe useState

Listing 4 – Syntaxe de base useState

```
import { useState } from 'react';

function Counter() {
  // Déclaration state
  const [count, setCount] = useState(0);
  //      ^           ^           ^
  //      |           |           |
  //  Valeur  Fonction  Valeur initiale
  //      pour modifier

  return (
    <div>
      <p>Compteur : {count}</p>
      <button onClick={() => setCount(count + 1)}>
        Incrémenter
      </button>
    </div>
  );
}
```

Décomposition :

- **count** : Variable state actuelle
- **setCount** : Fonction pour modifier count
- **useState(0)** : Valeur initiale = 0

△ Important

Règle absolue : Ne JAMAIS modifier state directement

```
// INTERDIT  
count = count + 1; // Ne marche pas, pas de re-render  
  
// CORRECT  
setCount(count + 1); // Déclenche re-render
```

3.3 Exemple Complet Compteur

Listing 5 – Compteur avec boutons

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  const increment = () => {
    setCount(count + 1);
  };

  const decrement = () => {
    setCount(count - 1);
  };

  const reset = () => {
    setCount(0);
  };

  return (
    <div className="counter">
      <h2>Compteur</h2>
      <p className="count-display">{count}</p>

      <div className="buttons">
        <button onClick={increment}>+</button>
        <button onClick={decrement}>-</button>
        <button onClick={reset}>Reset</button>
      </div>
    </div>
  );
}

export default Counter;
```

3.4 Types de Valeurs State

Le state peut contenir n'importe quel type :

Listing 6 – Différents types de state

```
// Number
const [count, setCount] = useState(0);

// String
const [name, setName] = useState('');

// Boolean
const [isVisible, setIsVisible] = useState(false);

// Array
const [items, setItems] = useState([]);

// Object
const [user, setUser] = useState({
  name: 'Ahmed',
  age: 20
});
```

```
});
```

3.5 State vs Props

Props	State
Reçues du parent	Gérées localement
Immuables (read-only)	Mutables (setState)
Communication descendante	Données internes
Passées via attributs JSX	Déclarées avec useState
Ne déclenchent pas re-render	Déclenchent re-render

Listing 7 – Props vs State en action

```
// Props : Parent passe données
function App() {
  return <ProductCard name="Laptop" price={5000} />;
}

function ProductCard({ name, price }) { // Props
  // State : Gérer compteur interne
  const [quantity, setQuantity] = useState(1);

  return (
    <div>
      <h3>{name}</h3> /* Props */
      <p>{price} DH</p> /* Props */
      <p>Quantité : {quantity}</p> /* State */
      <button onClick={() => setQuantity(quantity + 1)}>
        +
      </button>
    </div>
  );
}
```

★ Astuce

Quand utiliser State vs Props ?

Props : Données venant du parent (nom produit, prix)

State : Données qui changent (quantité, formulaire, toggle)

PAUSE COURS - Exercice Papier 1 (useState)

Durée : 20 minutes

Exercice : Créer composant `LikeButton` avec :

- State `likes` initialisé à 0
- Bouton "J'aime" qui incrémente likes
- Affichage : "X j'aime(s)"

Bonus : Ajouter bouton "Je n'aime plus" qui décrémente (minimum 0)